



ASSIGNMENT 1

SELF-REFLECTIVE REPORT

OBJECT ORIENTED PROGRAMMING

Lecturer: Sir Nazmirul Izzad bin Nassir

NAME	STUDENT ID	CLASS CODE	PROGRAM	PASSPORT
ALI SHARIF ABDULKADIR SHARIF	(202505010493)	BIT1123	BCSSE	[REDACTED]

GitHub Repository: <https://github.com/SHAFOO11/ALI-SHARIF-ABDULKADIR-SHARIF--202505010493-JAVA-PROGRAMMING>

1. Introduction:

The report captures my learning process throughout the ten tutorial classes of Object-Oriented Programming.

With each week passing through the semester, the assignments moved from just one `System.out.println` statement in Week 1 to a complete event-driven Swing desktop application in Week 10 and putting everything together into one place provided me with a clear picture of this process.

What changed since before this course was my perspective on programming as an activity: previously I viewed a program as an ordered sequence of commands to be performed sequentially from top to bottom. What changed since then is that I start with thinking about what objects are involved in the task, their roles, interaction between them and only after that move on to writing actual statements.

This report documents my findings throughout the tutorials, the exact problems that appeared during the process, ways to overcome them, and my future.

2. Knowledge Gained from Each Tutorial:

Week 1: Java Basics. Language specifics: primitives, operators, conditionals, iterations and arrays. Working on the `StudentGrade.java` file showed me that there is more to programming than just syntax, since separating grade calculation functionality to its own `calculateGrade()` method rather than using the if-else chain in the loop was the first example of separating a responsibility.

Week 2: Classes and Objects. The shift from data to modelled data. Defining `Student` with attributes, a constructor and behaviour, then creating two instances in `Mean.java`, made the distinction between a class and an object concrete: one blueprint, many independent objects each holding their own state.

Week 3–4: Inheritance and Polymorphism. The pivotal tutorial. `Person` was extended by `Student` and `Lecturer`, each overriding `introduce()`. Declaring all three as `Person` references and calling the same method on each and getting three different outputs is what made runtime polymorphism click for me.

Week 5: Encapsulation. Making all fields private and accessing them through getter/setter methods. What we learned from this was that having a public interface to a class is a design choice.

Week 6: Protected access and inheritance. The employee has used protected access to allow the Lecturer to use those fields directly, even though they would be inaccessible to other classes. This covered the middle ground that was missing in Week 5.

Week 7: Abstraction. Appliance as an abstract class with shared concrete behavior plus an abstract operate () that each subclass had to implement. Seeing WashingMachine, Refrigerator and AirConditioner all driven through a single Appliance reference showed me the value of programming to a supertype rather than to concrete classes.

Week 8-9: Collections, File I/O and Exceptions. ArrayList collection type that expands automatically, BufferedReader and BufferedWriter classes for file persistence, and try-with-resources statement to ensure that streams get closed even if something goes wrong. This tutorial gave the first notion that it's normal situation for software to fail and that we must handle it.

Week 10: Swing GUI Mini Project. Creating Code Boss Battle involved nesting panels, setting up ActionListener via lambdas, creating animations with javax.swing.Timer, and ensuring consistency of game state for each user input. It is vital to note that this task was all about splitting the model (Questions) from the view (QuizBattleGUI), which was the first time I had split them.

3. Challenges Encountered:

Polymorphism. During Week 3–4 I could not initially understand why `Person p2 = new Student(...)` made sense. Since the variable was declared as `Person`, why would we not just make it a student? The reason for such an approach was not clear to me.

Package declarations that did not match their folders. Several tutorials declared packages such as `package week_3_4;` inside a folder named `week_3-4`. The files compiled but would not run with a plain java Main, and the error messages about the class not being found did not point at the real cause.

Deciding what belongs in a class. In Week 6 I was unsure whether subject and department belonged on `Employee` or on `Lecturer`. Both placements compiled and ran, so the compiler gave me no signal about which was the better design.

Abstract classes felt like extra ceremony. In Week 7 my instinct was that if `operate ()` has no body, it should simply be left out. Writing a method signature with no implementation seemed like wasted effort.

Swing's complexity in Week 10. The GUI was by far the hardest tutorial. Layout managers did not place components where I expected, and my first version put question data, game rules and interface code together in one class which became very difficult to change.

Code that only worked on my machine: Two problems surfaced only when the project was run elsewhere. QuizBattleScreenshot.java contained an absolute path (/workspaces/...) that existed on no other computer, and the boss character was rendered with Segoe UI Emoji, a Windows font, so on other systems it displayed as an empty box. Both worked perfectly where I wrote them, which is exactly why I did not notice.

Weak version control habits: Reviewing the repository history, most of my commit messages read simply done. When I needed to find where a specific change had been introduced, the history told me almost nothing.

4. How the Challenges Were Overcome:

Polymorphism became clear through deliberate experiment rather than re-reading notes. I changed p2.introduce() to call the student version, then the Lecturer version, and watched the output change while the declaring type stayed Person. Seeing that Java resolves the call against the actual object at runtime not the declared reference type is what made the concept concrete.

The package mismatches I resolved by removing the package declarations entirely, so each week folder is self-contained in the default package. Every folder now compiles and runs with an identical pair of commands, which also made the "How to Run" section of the README straightforward to write. The wider lesson was that the compiler accepting my code is not the same as my code being correct.

For the class design question in Week 6, I applied a simple test: does every Employee have a subject? A receptionist does not, so subject belongs on Lecturer. Asking whether an attribute is true of *every* instance of the base class became my default check.

Abstract classes made sense once I deliberately broke one. I removed operate() from a subclass and the compiler refused to build which was the point. The abstract method is a contract enforced at compile time, and that guarantee is what lets Main call operate() on any Appliance with confidence.

For the class design question in Week 6, I applied a simple test: does every Employee have a subject? A receptionist does not, so subject belongs on Lecturer. Asking whether an attribute is true of every instance of the base class became my default check.

Abstract classes made sense once I deliberately broke one. I removed operate() from a subclass and the compiler refused to build which was the point. The abstract method is a contract enforced at compile time, and that guarantee is what lets Main call operate() on any Appliance with confidence.

Swing I implemented through splitting up the UI construction into distinct builder functions `buildTopPanel()`, `buildBossPanel()`, `buildBottomPanel()` rather than putting everything in the constructor.

The question data was then extracted into its own `Questions` class with a static factory method. Having the model separated from the view, switching out the question pool required no changes to the UI code at all.

The machine-dependent issues I fixed by switching the hardcoded path to a relative one provided as a parameter, and by substituting the emoji with a text monospace character depiction that displays identically regardless of which system you're running on. In addition, I added a check for a headless environment to switch to a console implementation in case there's no display available. It turns out, testing your program on a different OS reveals a whole new class of bugs that would not be found otherwise.

Regarding version control, I now commit changes with a message describing the change what it is and why it's done.

5. Improvements in Java Programming Skills:

The clearest technical improvements are these. I can compile and run multi-class Java programs from the command line and read compiler errors as useful diagnostics rather than obstacles. I use private fields with accessors by default instead of exposing state directly. I can build a class hierarchy that uses inheritance because the relationship genuinely is "is-a", not merely to avoid retyping code.

Beyond the language itself, three habits changed. I now split responsibilities across separate files rather than growing one large class. Week 7 has five focused files and is far easier to follow than a single combined one would be. I write code that anticipates failure, using `try-with-resources` and handling `IOException` explicitly. And I check that a program works outside my own environment, which is a discipline the Week 10 font and path bugs taught me directly.

My debugging approach also matured. Early in the semester I would change lines until the error disappeared. Now I read the error, form a hypothesis about the cause, and test that hypothesis a much faster process, and one that leaves me understanding the fix.

6. Understanding of Object-Oriented Programming Concepts:

Encapsulation refers to keeping a class internal state private while providing a controlled public interface. The advantage of encapsulation is that any changes to the internals will not affect any code that uses the class since nothing outside the class is dependent on them. In the case of Student in Week 5, all fields are completely inaccessible outside the class, and all interaction happens only through methods in the class.

Inheritance allows for a subclass to inherit fields and behavior from a superclass and build upon them, thus providing an "is-a" relationship like a Lecturer being an Employee. It's true power lies not in reducing typing but in having a common type, allowing for treating different objects uniformly. Another thing that I learned about it is where it does not apply, namely in cases of "Has-a".

The benefit of polymorphism is that the same interface can be used to call many implementations, which is why the exact same call will yield the correct behavior based on what actual object is there. Week 3-4 exemplifies this concept in the clearest way: three Person objects, one call, three different results, determined at runtime. Week 7 uses the same concept effectively Main method calls operate (), regardless of what kind of appliance it has.

The meaning of abstraction is that we expose the functionality of an object, without exposing how this functionality is performed. Appliance class states that all appliances must have operate (), and each subclass figures out what exactly it means. Abstraction is what enables us to design our programs to rely on contract instead of implementation.

Working with the consolidated repository opened my eyes that these four principles were not separate ways of doing things, but rather one coherent strategy of designing a system: encapsulation defines the boundaries, inheritance and abstraction define relations between them, and polymorphism is what makes these relations work.

7. Future Learning Plans:

In the short term I want to strengthen the fundamentals this course opened: interfaces and multiple inheritance of type, generics beyond the basic ArrayList<String> usage in Week 8-9, and the wider Collections Framework HashMap, Set and the differences between implementations.

Next, I intend to learn unit testing with JUnit. Every tutorial in this repository was verified by running it and reading the output, which does not scale and does not protect against regressions. Automated tests would have caught the Week 10 path bug immediately.

Beyond that, my goals are to study common design patterns Factory, Observer, Strategy since Questions.loadQuestions() is effectively a static factory I arrived at by intuition rather than by knowing the pattern; to deepen my Git practice with branching and pull requests instead of committing directly to main; and to build a larger project that connects a Java application to a database, so that data persists in something more capable than a text file.

Longer term I want to explore Java for backend development with Spring Boot, and to apply object-oriented design in another language to see which principles are universal and which are specific to Java.

8. Conclusion:

most valuable outcome of this course is not any single language feature but a change in how I approach a problem. I now start by asking what objects the problem contains and what each should be responsible for, and I have found that decisions made at that level determine how difficult everything afterwards becomes.

Consolidating ten weeks of tutorials into one repository was itself instructive. Reviewing the earlier work with what I know now, I could see clearly where the design was weak the empty Week 1 files, the mismatched package declarations, the machine-specific paths and I was able to fix them. Being able to critique my own earlier code is, I think, the clearest evidence that the understanding is real.

The Week 10 project brought everything together. Code Boss Battle required encapsulation for the question model, composition for the interface, polymorphism through event handlers, exception handling for file operations, and a deliberate separation between model and view. None of that was possible in Week 1, and being able to build it is a concrete measure of what the semester produced.